PRACTICAL FILE

Advanced DBMS and Administration

(Paper 1T4)

First Year M.C.A. — Semester I (CBCS)

RTM Nagpur University

Name: _____

Roll No: _____

INDEX

Sr. No.	Practical	Date	Sign
1	Create a new user & grant privileges of creating session, table & view to this new user also check connect or not.		
2	Connect to the new user, assign a tablespace, verify it works, create a table, and insert data.		
3	Create a new role named manager with same privileges. Grant this role to new user. Then change the password of newly created user.		
4	Revoke the privileges from role manager. Revoke role from user then delete that user.		
5	Create a tablespace named “P_student” with file name “Stu_datafile.dbf” having size of 10mb.		
6	Create a tablespace named “ts_mydemo” with two datafile named “ts_mydemo01.dbf” & “ts_mydemo02.dbf” having size of 50mb and 64mb respectively. Make the tablespace auto extendable and limit auto extending to size of 2048mb. In auto extending, next extend should of 32mb.		
7	Alter the last datafile “ts_mydemo01.dbf” so that its size will automatically extend by 5mb & limit its size to 500mb.		
8	Create temporary table space with size of 5mb which will be automatically increase on demand.		
9	Add new Redo log file of size 10mb to existing database.		
10	Display the list of all Redo log files and Control files.		
11	Display the list, datafiles, tablespace names & size in MB.		
12	Create rollback segment in separate tablespace with proper storage option.		
13	Create a 3 new abstract datatype as given below: 1. NAME_TYPE(F_NAME, M_NAME, L_NAME) 2. POST_TYPE(MAIN, SFX) 3. ADDR_TYPE(STREET, CITY, STATE, POSTCODE)		
14	Create a table stud_info having column name, rollno, phy, maths, comp, total and percentage. Write a trigger to automatically calculate students total marks and percentage of students while insertion.		
15	Write a SQL query to display the following data information: a. Name of database b. Database startup time c. Database size information (database size, used space, free space)		

Practical No. 1

Aim: Create a new user & grant privileges of creating session, table & view to this new user also check connect or not.

*Run as SYSTEM (DBA) user in SQL*Plus.*

Algorithm:

Step 1: Start.

Step 2: Login to SQL*Plus as SYSTEM (DBA) user.

Step 3: Create a new user Prerna with password prerna123 using CREATE USER.

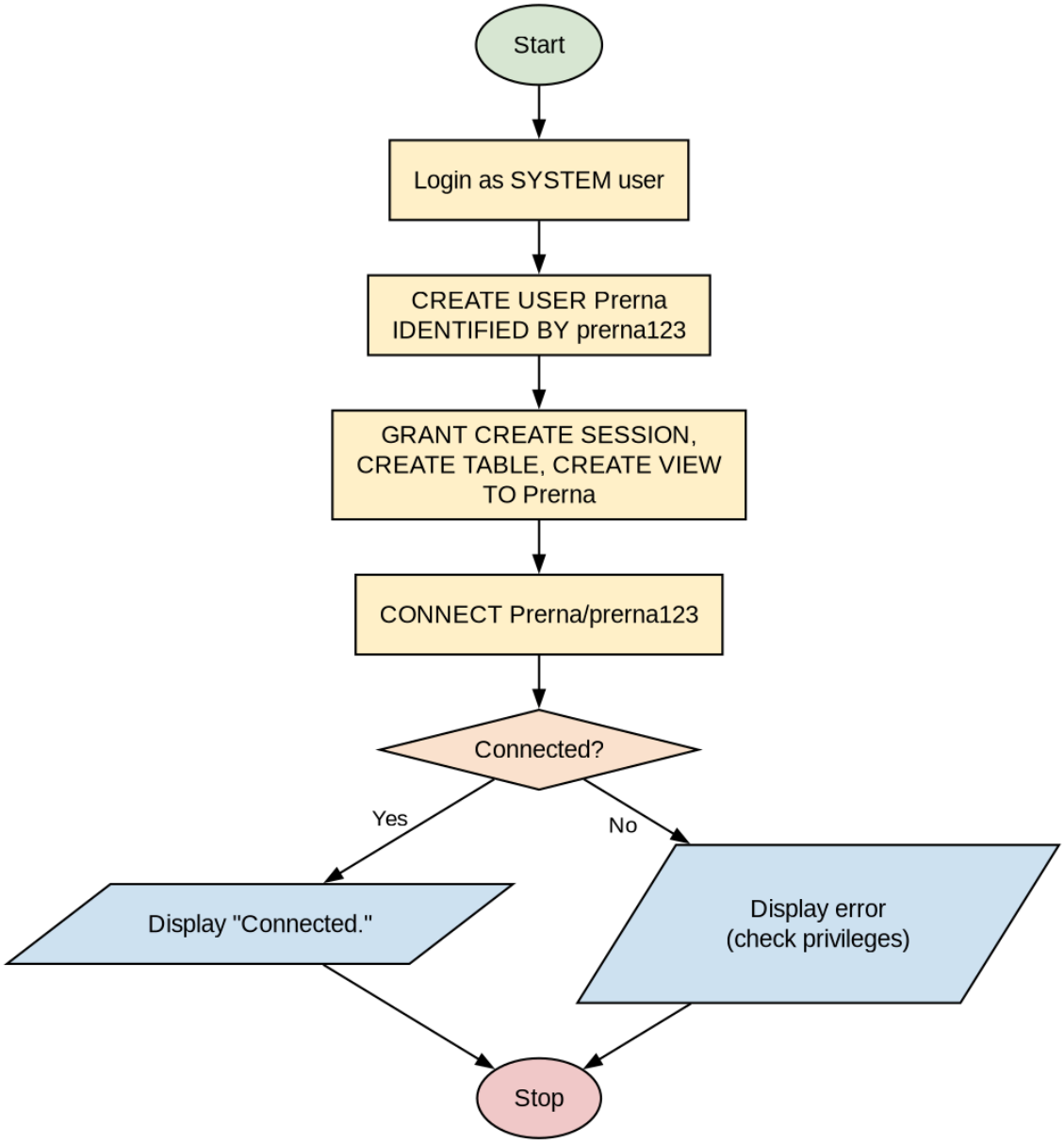
Step 4: Grant CREATE SESSION, CREATE TABLE and CREATE VIEW privileges to the user using GRANT.

Step 5: Connect as the new user using CONNECT Prerna/prerna123.

Step 6: If the message "Connected." is displayed, the user is created and working properly; otherwise check the privileges.

Step 7: Stop.

Flowchart:


Start
Login as SYSTEM user
CREATE USER Prerna
IDENTIFIED BY prerna123
GRANT CREATE SESSION,
CREATE TABLE, CREATE VIEW
TO Prerna
CONNECT Prerna/prerna123
Connected?
Yes
No
Display "Connected."
Display error
(check privileges)
Stop

Program (SQL Queries):

```
-- Create user
CREATE USER Prerna IDENTIFIED BY prerna123;

-- Grant privileges to user
GRANT CREATE TABLE, CREATE SESSION, CREATE VIEW TO Prerna;

-- Connect user
CONNECT Prerna/prerna123;
```

Output:

```
-- Create user
SQL> create user Prerna identified by prerna123;

User created.

-- Grant privileges to user
SQL> grant create table, create session, create view to Prerna;

Grant succeeded.

-- Connect user
SQL> CONNECT Prerna/prerna123;

Connected.
```

Practical No. 2

Aim: Connect to the new user, assign a tablespace, verify it works, create a table, and insert data.

Algorithm:

Step 1: Start.

Step 2: As SYSTEM user, assign DEFAULT TABLESPACE users, TEMPORARY TABLESPACE temp and QUOTA 10M on users to user Prerna using ALTER USER.

Step 3: Connect as user Prerna.

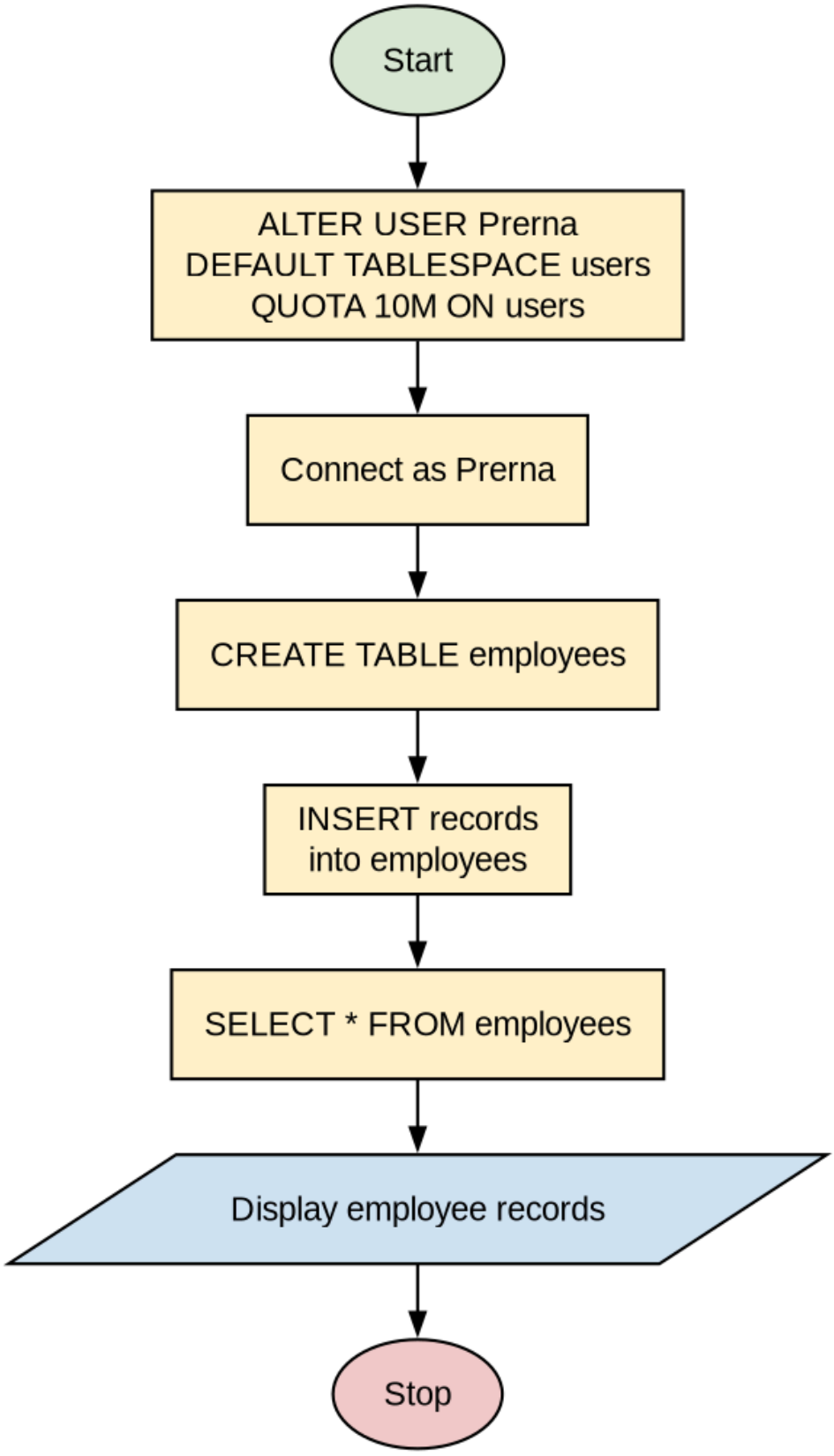
Step 4: Create table employees (emp_id, emp_name, salary).

Step 5: Insert records into the employees table using INSERT INTO.

Step 6: Display the data using SELECT * FROM employees to verify the tablespace works.

Step 7: Stop.

Flowchart:


Start
ALTER USER Prerna
DEFAULT TABLESPACE users
QUOTA 10M ON users
Connect as Prerna
CREATE TABLE employees
INSERT records
into employees
SELECT * FROM employees
Display employee records
Stop

Program (SQL Queries):

```
-- Tablespace privilege
ALTER USER Prerna
    DEFAULT TABLESPACE users
    TEMPORARY TABLESPACE temp
    QUOTA 10M ON users;

-- Connect user
CONN Prerna/prerna123;

-- Create table
CREATE TABLE employees (
    emp_id   NUMBER PRIMARY KEY,
    emp_name VARCHAR2(50),
    salary   NUMBER(10,2)
);

-- Insert data in table
INSERT INTO employees VALUES (1, 'John Doe', 50000);
INSERT INTO employees VALUES (2, 'Jane Smith', 60000);

-- Show data
SELECT * FROM employees;
```

Output:

```
SQL> ALTER USER Prerna
  2      DEFAULT TABLESPACE users
  3      TEMPORARY TABLESPACE temp
  4      QUOTA 10M ON users;
User altered.

SQL> conn
Enter user-name: Prerna
Enter password:
Connected.

SQL> CREATE TABLE employees (
  2      emp_id NUMBER PRIMARY KEY,
  3      emp_name VARCHAR2(50),
  4      salary NUMBER(10,2)
  5  );
Table created.

SQL> INSERT INTO employees VALUES (1, 'John Doe', 50000);
1 row created.
SQL> INSERT INTO employees VALUES (2, 'Jane Smith', 60000);
1 row created.

SQL> SELECT * FROM employees;

    EMP_ID EMP_NAME                  SALARY
---------- -------------------- ----------
         1 John Doe                  50000
         2 Jane Smith                60000
```

Practical No. 3

Aim: Create a new role named manager with same privileges. Grant this role to new user. Then change the password of newly created user.

Algorithm:

Step 1: Start.

Step 2: Create a role named manager using CREATE ROLE.

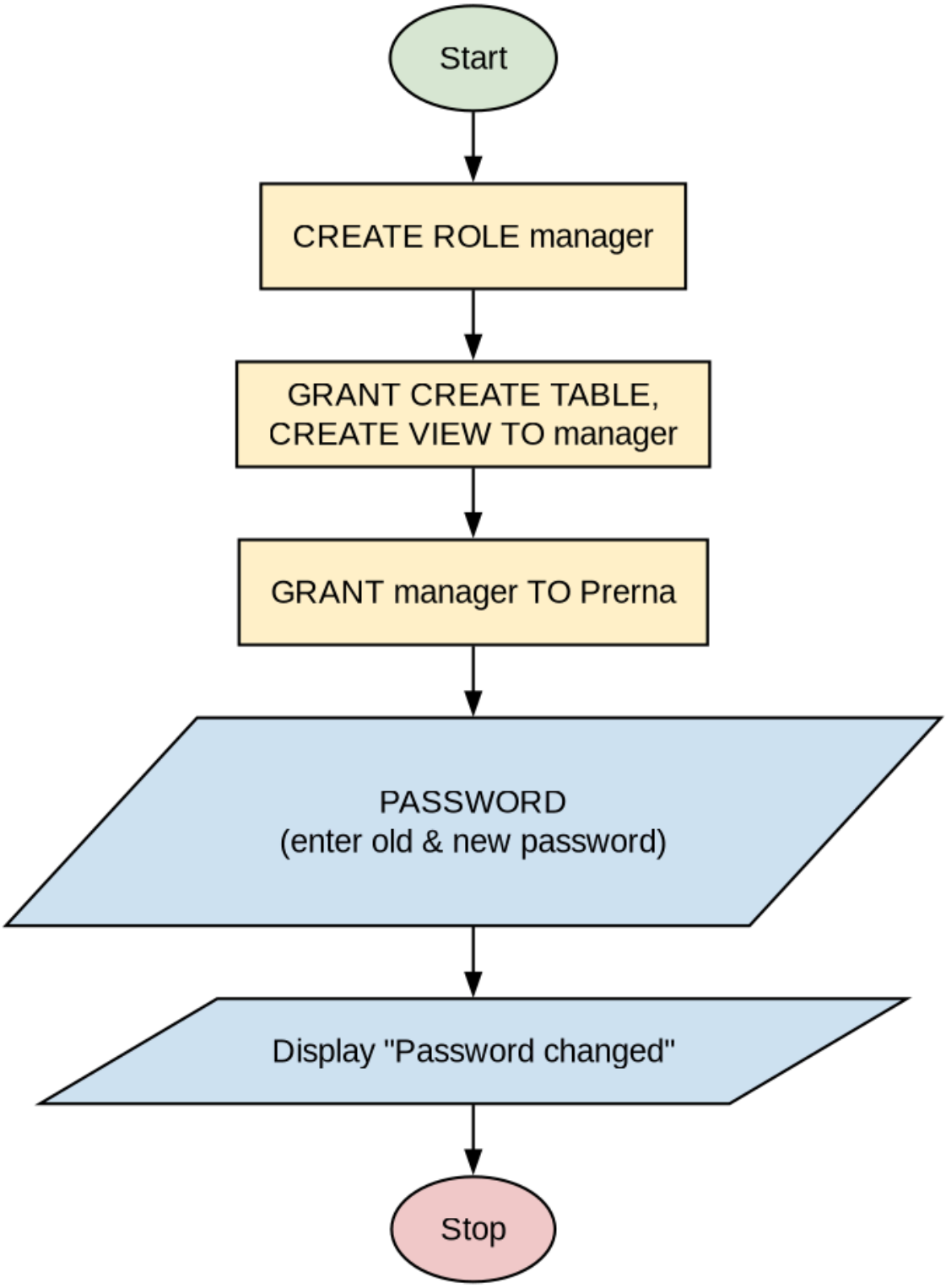
Step 3: Grant CREATE TABLE and CREATE VIEW privileges to role manager.

Step 4: Grant the role manager to user Prerna.

Step 5: Change the password of user Prerna using the PASSWORD command (enter old and new password).

Step 6: Stop.

Flowchart:


Start
CREATE ROLE manager
GRANT CREATE TABLE,
CREATE VIEW TO manager
GRANT manager TO Prerna
PASSWORD
(enter old & new password)
Display "Password changed"
Stop

Program (SQL Queries):

```
-- Create role
CREATE ROLE manager;

-- Grant privileges to role
GRANT CREATE TABLE, CREATE VIEW TO manager;

-- Assign role to user
GRANT manager TO Prerna;

-- Change password
PASSWORD
```

Output:

```
-- Create role
SQL> create role manager;

Role created.

-- Grant privileges to role
SQL> grant create table, create view to manager;

Grant succeeded.

-- Assign role to user
SQL> grant manager to Prerna;

Grant succeeded.

-- Change password
SQL> password
Changing password for PRERNA
Old password:
New password:
Retype new password:
Password changed
```

Practical No. 4

Aim: Revoke the privileges from role manager. Revoke role from user then delete that user.

Algorithm:

Step 1: Start.

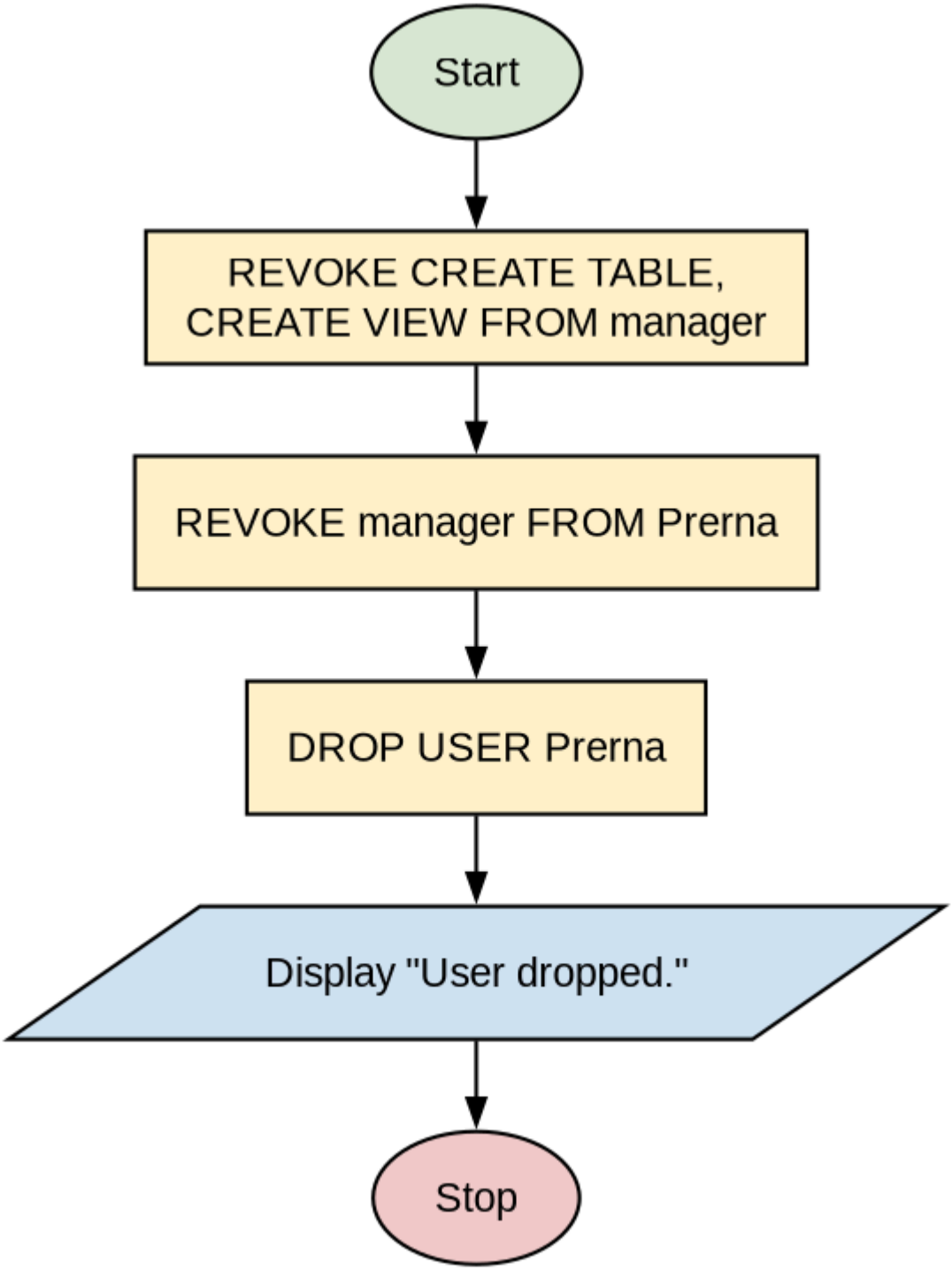
Step 2: Revoke CREATE TABLE and CREATE VIEW privileges from role manager using REVOKE.

Step 3: Revoke the role manager from user Prerna.

Step 4: Delete the user Prerna using DROP USER.

Step 5: Stop.

Flowchart:


Start
REVOKE CREATE TABLE,
CREATE VIEW FROM manager
REVOKE manager FROM Prerna
DROP USER Prerna
Display "User dropped."
Stop

Program (SQL Queries):

```
-- Revoke privileges
REVOKE CREATE TABLE, CREATE VIEW FROM manager;

-- Revoke role from user
REVOKE manager FROM Prerna;

-- Delete user
DROP USER Prerna;
```

Output:

```
-- Revoke privileges
SQL> Revoke create table, create view from manager;

Revoke succeeded.

-- Revoke role from user
SQL> Revoke manager from Prerna;

Revoke succeeded.

-- Delete user
SQL> drop user Prerna;

User dropped.
```

Practical No. 5

Aim: Create a tablespace named “P_student” with file name “Stu_datafile.dbf” having size of 10mb.

Algorithm:

Step 1: Start.

Step 2: Grant the CREATE TABLESPACE privilege to the user.

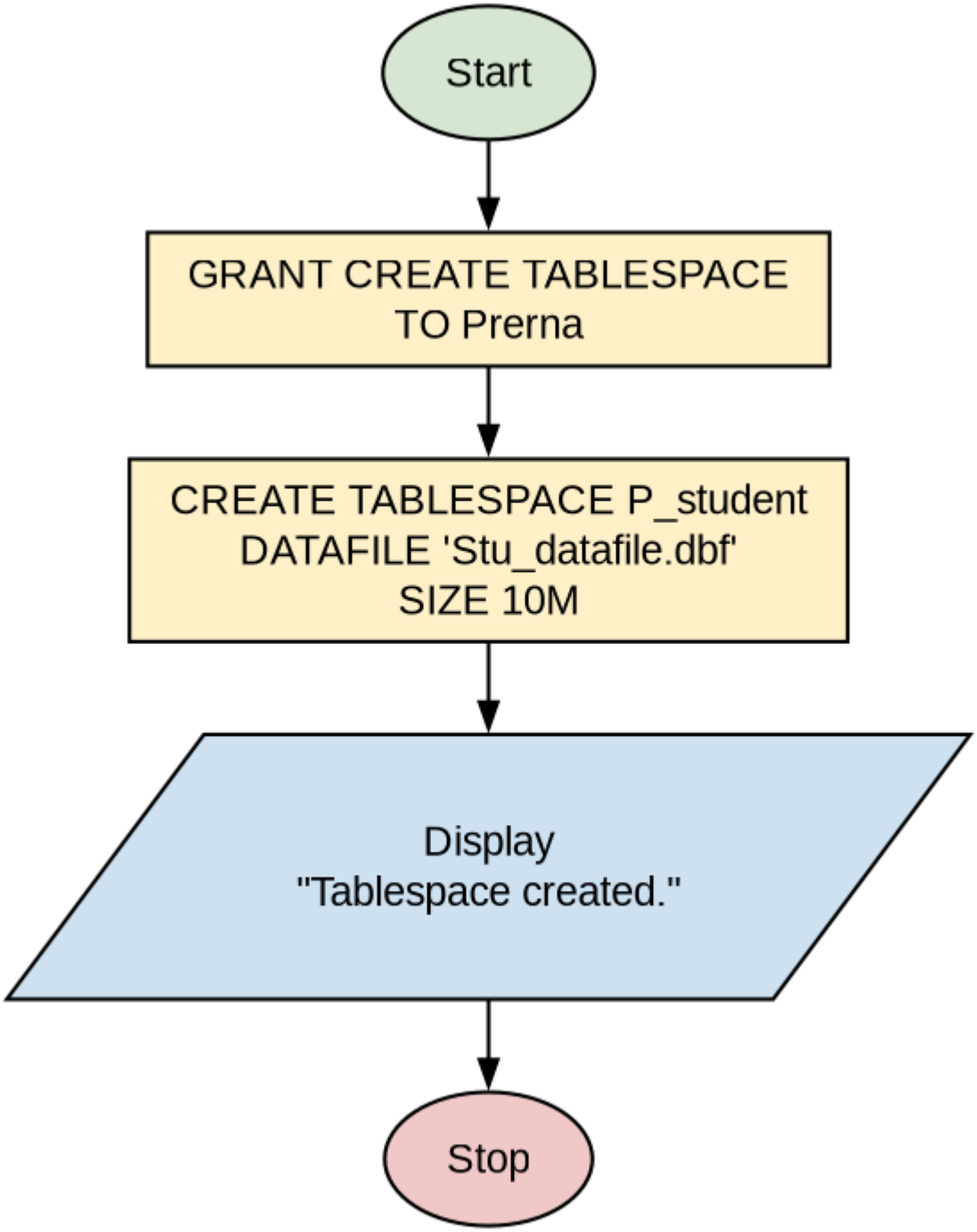
Step 3: Create tablespace P_student with DATAFILE 'Stu_datafile.dbf' of SIZE 10M using CREATE TABLESPACE.

Step 4: Display the message "Tablespace created."

Step 5: Stop.

Note: *The answer is corrected here: the tablespace name is changed from Std_student (answer sheet) to P_student as given in the practical list. Please test and run it yourself before writing.*

Flowchart:


Start
GRANT CREATE TABLESPACE
TO Prerna
CREATE TABLESPACE P_student
DATAFILE 'Stu_datafile.dbf'
SIZE 10M
Display
"Tablespace created."
Stop

Program (SQL Queries):

```
-- Grant privilege
GRANT CREATE TABLESPACE TO Prerna;

-- Create tablespace
CREATE TABLESPACE P_student
    DATAFILE 'Stu_datafile.dbf'
    SIZE 10M;
```

Output:

```
-- Grant privilege
SQL> GRANT CREATE TABLESPACE TO Prerna;

Grant succeeded.

-- Create tablespace
SQL> CREATE TABLESPACE P_student
  2  DATAFILE 'Stu_datafile.dbf'
  3  SIZE 10M;

Tablespace created.
```

Practical No. 6

Aim: Create a tablespace named “ts_mydemo” with two datafile named “ts_mydemo01.dbf” & “ts_mydemo02.dbf” having size of 50mb and 64mb respectively. Make the tablespace auto extendable and limit auto extending to size of 2048mb. In auto extending, next extend should of 32mb.

Algorithm:

Step 1: Start.

Step 2: Use CREATE TABLESPACE ts_mydemo.

Step 3: Specify the first datafile of SIZE 50M with AUTOEXTEND ON NEXT 32M MAXSIZE 2048M.

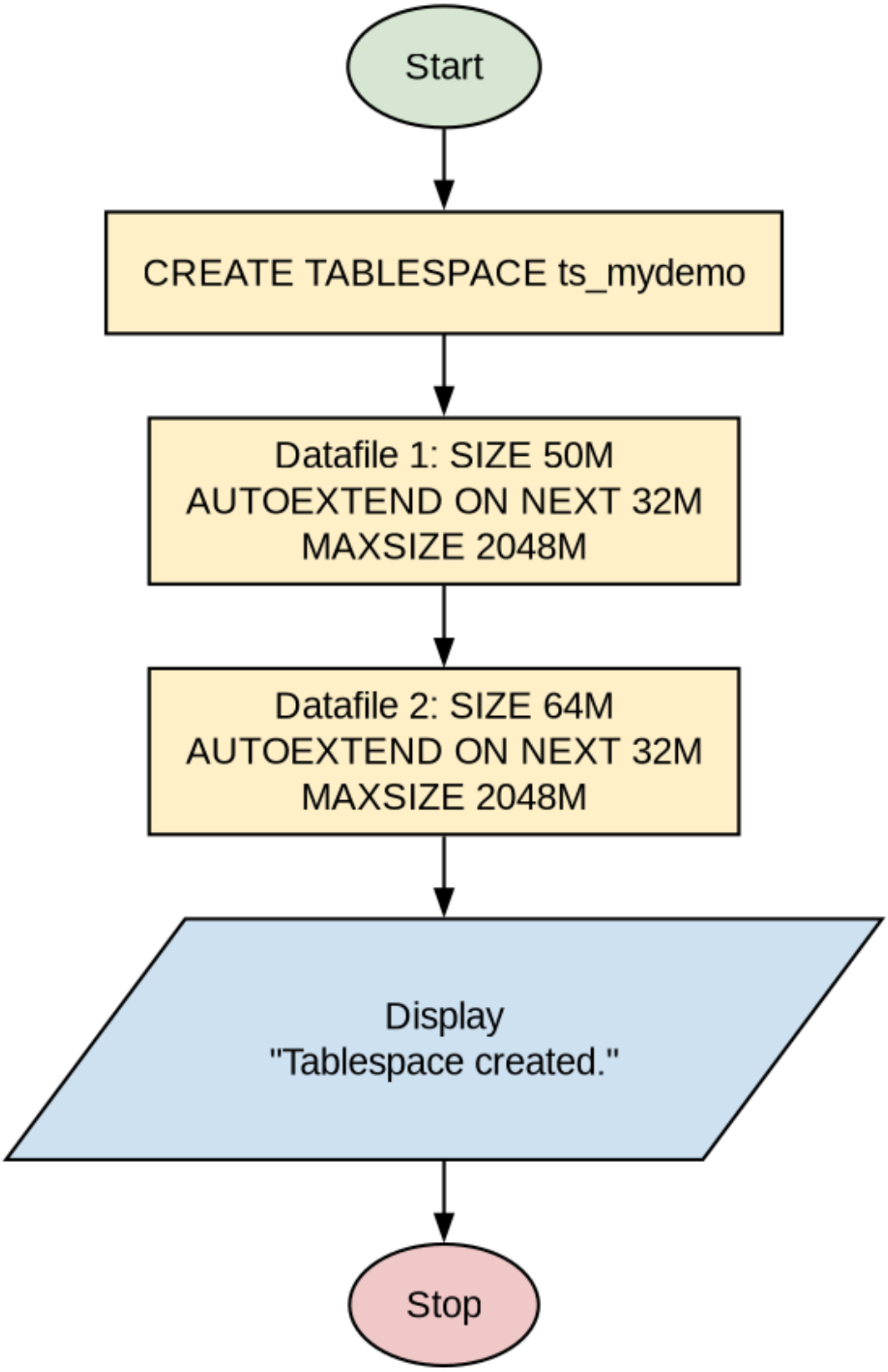
Step 4: Specify the second datafile of SIZE 64M with AUTOEXTEND ON NEXT 32M MAXSIZE 2048M.

Step 5: Execute the statement and display "Tablespace created."

Step 6: Stop.

Note: *The answer is corrected here: the datafile names are changed from mydemo01.dbf / mydemo02.dbf (answer sheet) to ts_mydemo01.dbf / ts_mydemo02.dbf as given in the practical list. Please test and run it yourself before writing.*

Flowchart:


Start
CREATE TABLESPACE ts_mydemo
Datafile 1: SIZE 50M
AUTOEXTEND ON NEXT 32M
MAXSIZE 2048M
Datafile 2: SIZE 64M
AUTOEXTEND ON NEXT 32M
MAXSIZE 2048M
Display
"Tablespace created."
Stop

Program (SQL Queries):

```
-- Create tablespace with two datafiles
CREATE TABLESPACE ts_mydemo
    DATAFILE 'ts_mydemo01.dbf' SIZE 50M
        AUTOEXTEND ON NEXT 32M
        MAXSIZE 2048M,
    'ts_mydemo02.dbf' SIZE 64M
        AUTOEXTEND ON NEXT 32M
        MAXSIZE 2048M;
```

Output:

```
SQL> CREATE TABLESPACE ts_mydemo
  2  DATAFILE 'ts_mydemo01.dbf' SIZE 50M
     AUTOEXTEND ON NEXT 32M
     MAXSIZE 2048M,
  3  'ts_mydemo02.dbf' SIZE 64M
     AUTOEXTEND ON NEXT 32M
     MAXSIZE 2048M;

Tablespace created.
```

Practical No. 7

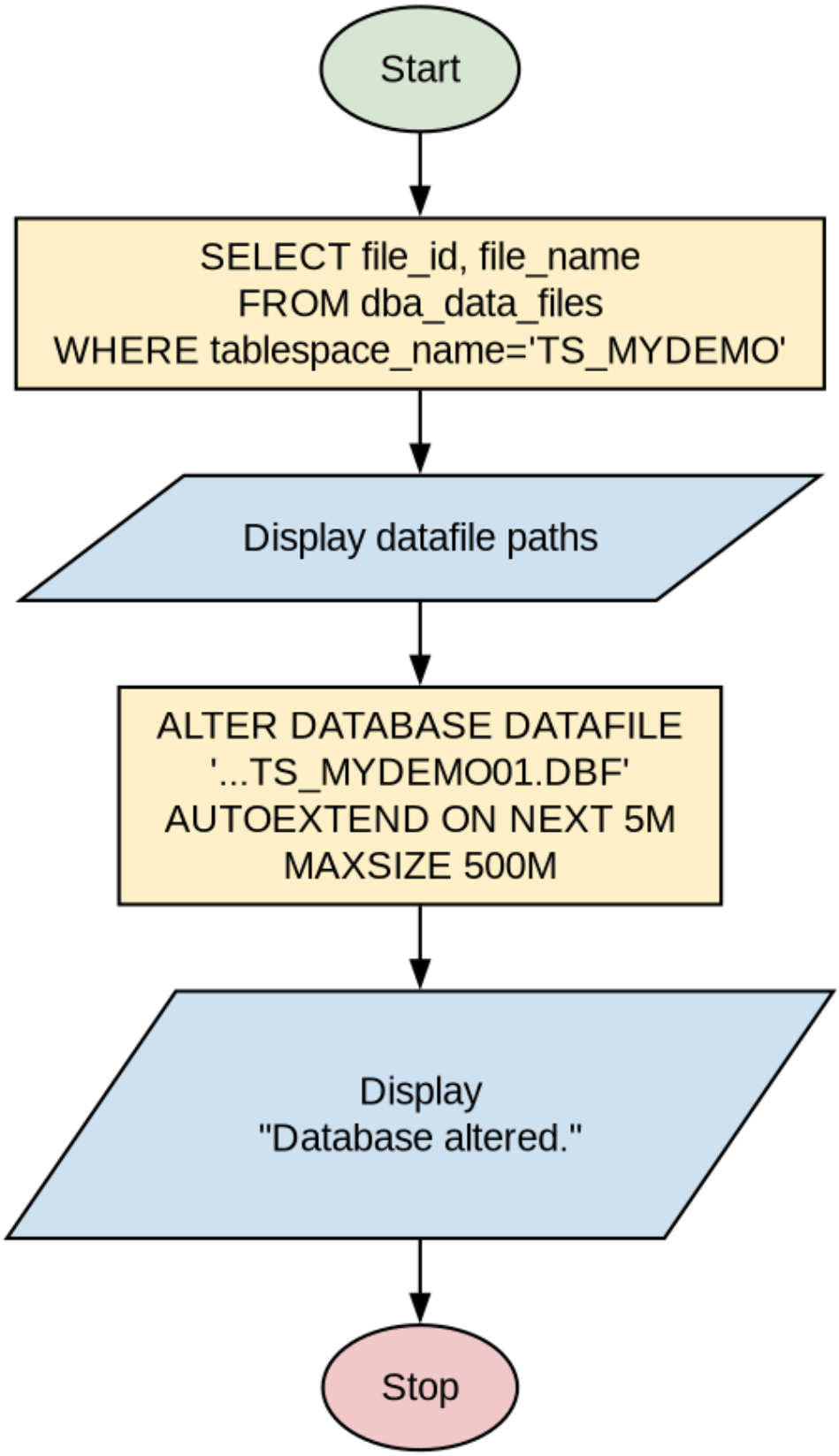
Aim: Alter the last datafile “ts_mydemo01.dbf” so that its size will automatically extend by 5mb & limit its size to 500mb.

Algorithm:

Step 1: Start.
Step 2: Find the full path of the datafiles of tablespace TS_MYDEMO from dba_data_files.
Step 3: Use ALTER DATABASE DATAFILE '<full path of TS_MYDEMO01.DBF>'.
Step 4: Set AUTOEXTEND ON NEXT 5M MAXSIZE 500M.
Step 5: Display the message "Database altered."
Step 6: Stop.

Note: *The datafile name/path is corrected to TS_MYDEMO01.DBF to match Practical 6. The full path depends on your Oracle installation. Please test and run it yourself before writing.*

Flowchart:


Start
SELECT file_id, file_name
FROM dba_data_files
WHERE tablespace_name='TS_MYDEMO'
Display datafile paths
ALTER DATABASE DATAFILE
'...TS_MYDEMO01.DBF'
AUTOEXTEND ON NEXT 5M
MAXSIZE 500M
Display
"Database altered."
Stop

Program (SQL Queries):

```
-- Find datafile path
SELECT file_id, file_name
FROM dba_data_files
WHERE tablespace_name = 'TS_MYDEMO';

-- Alter datafile
ALTER DATABASE
DATAFILE 'C:\ORACLEXE\APP\ORACLE\PRODUCT\11.2.0\SERVER\DATABASE\TS_MYDEMO01.DBF'
AUTOEXTEND ON
NEXT 5M
MAXSIZE 500M;
```

Output:

```
SQL> SELECT file_id, file_name
  2  FROM dba_data_files
  3  WHERE tablespace_name = 'TS_MYDEMO';

   FILE_ID FILE_NAME
---------- ------------------------------------------------------
        10 C:\ORACLEXE\APP\ORACLE\PRODUCT\11.2.0\SERVER\DATABASE\TS_MYDEMO01.DBF
        11 C:\ORACLEXE\APP\ORACLE\PRODUCT\11.2.0\SERVER\DATABASE\TS_MYDEMO02.DBF

-- Alter Datafile
SQL> ALTER DATABASE
  2  DATAFILE
'C:\ORACLEXE\APP\ORACLE\PRODUCT\11.2.0\SERVER\DATABASE\TS_MYDEMO01.DBF'
  3  AUTOEXTEND ON
  4  NEXT 5M
  5  MAXSIZE 500M;

Database altered.
```

Practical No. 8

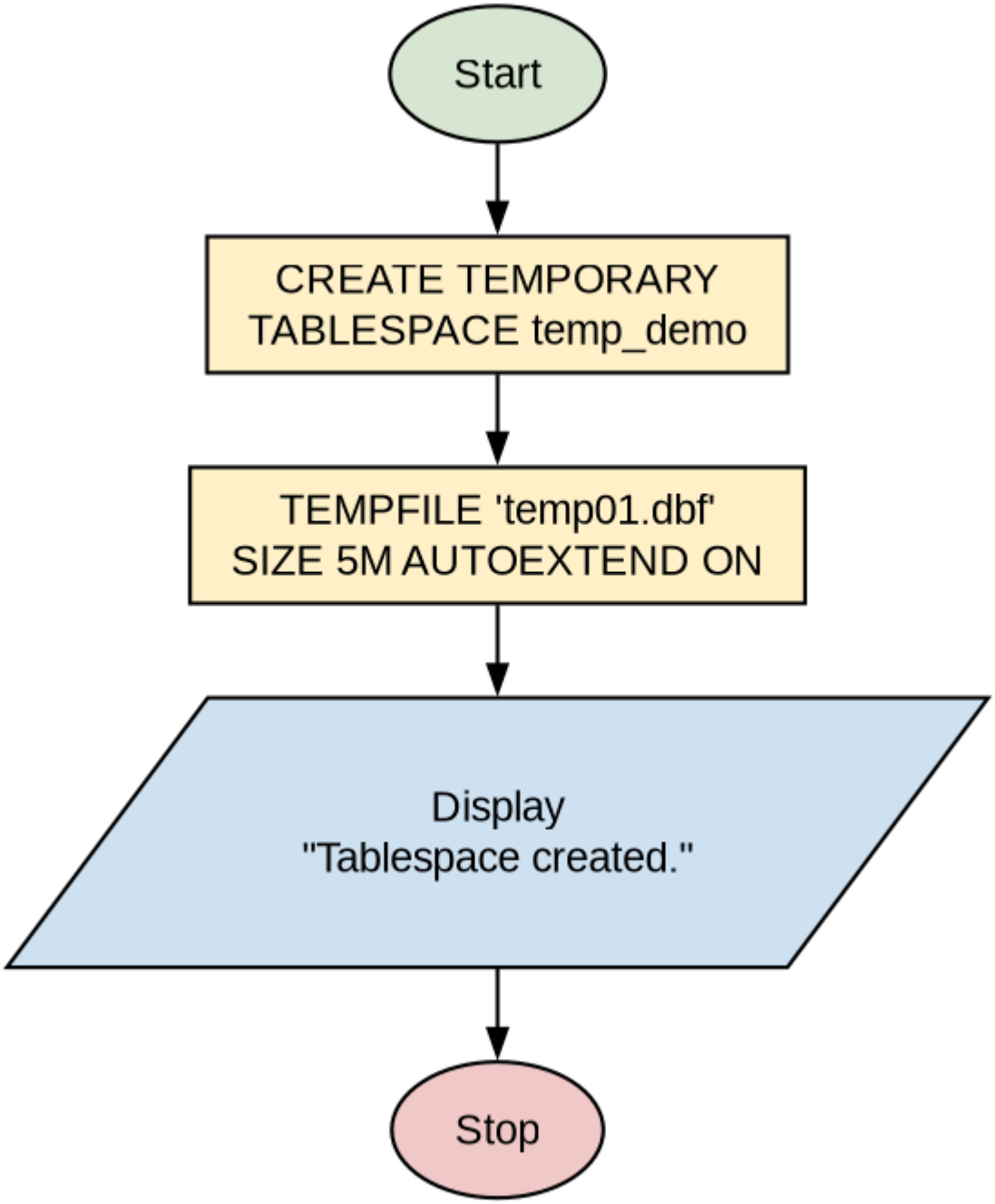
Aim: Create temporary table space with size of 5mb which will be automatically increase on demand.

Algorithm:

Step 1: Start.
Step 2: Use CREATE TEMPORARY TABLESPACE temp_demo.
Step 3: Specify TEMPFILE 'temp01.dbf' of SIZE 5M.
Step 4: Set AUTOEXTEND ON so that the size increases on demand.
Step 5: Display the message "Tablespace created."
Step 6: Stop.

Note: *The answer is corrected here: "temp file" and "auto extend" are written as TEMPFILE and AUTOEXTEND (correct Oracle syntax). Please test and run it yourself before writing.*

Flowchart:


Start
CREATE TEMPORARY TABLESPACE temp_demo
TEMPFILE 'temp01.dbf' SIZE 5M AUTOEXTEND ON
Display "Tablespace created."
Stop

Program (SQL Queries):

```
-- Create temporary tablespace
CREATE TEMPORARY TABLESPACE temp_demo
    TEMPFILE 'temp01.dbf' SIZE 5M AUTOEXTEND ON;
```

Output:

```
-- create tablespace
SQL> create temporary tablespace temp_demo
  2  tempfile 'temp01.dbf' size 5m autoextend on;

Tablespace created.
```

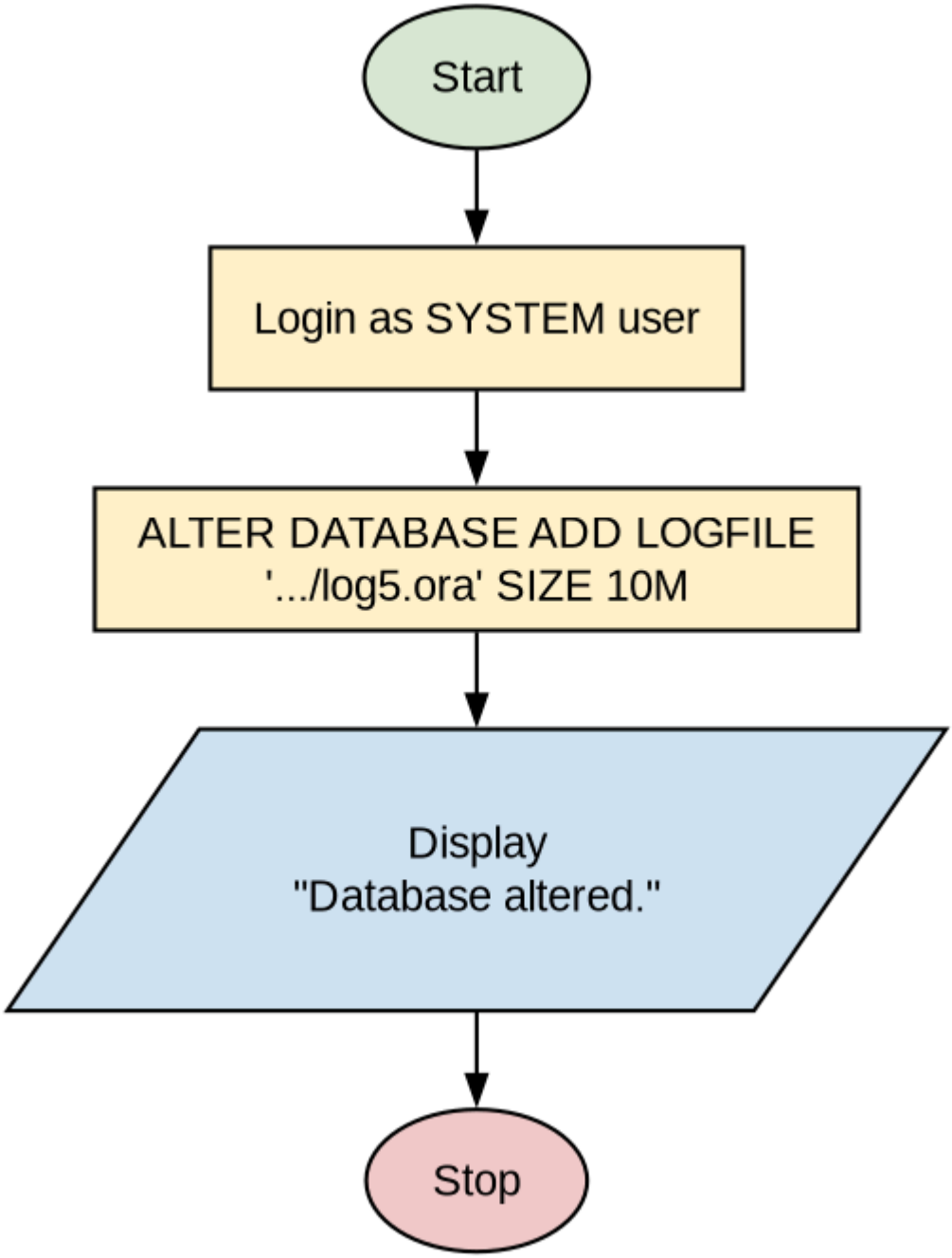
Practical No. 9

Aim: Add new Redo log file of size 10mb to existing database.

Algorithm:

Step 1: Start.
Step 2: Login as SYSTEM (DBA) user.
Step 3: Use ALTER DATABASE ADD LOGFILE with the file path and SIZE 10M.
Step 4: Display the message "Database altered."
Step 5: Stop.

Flowchart:


Start
Login as SYSTEM user
ALTER DATABASE ADD LOGFILE
'.../log5.ora' SIZE 10M
Display
"Database altered."
Stop

Program (SQL Queries):

```
-- Add new redo log file
ALTER DATABASE ADD LOGFILE 'c:/program files/oracle/log5.ora' SIZE 10M;
```

Output:

```
SQL> alter database add logfile 'c:/program files/oracle/log5.ora' size 10m;

Database altered.
```

Practical No. 10

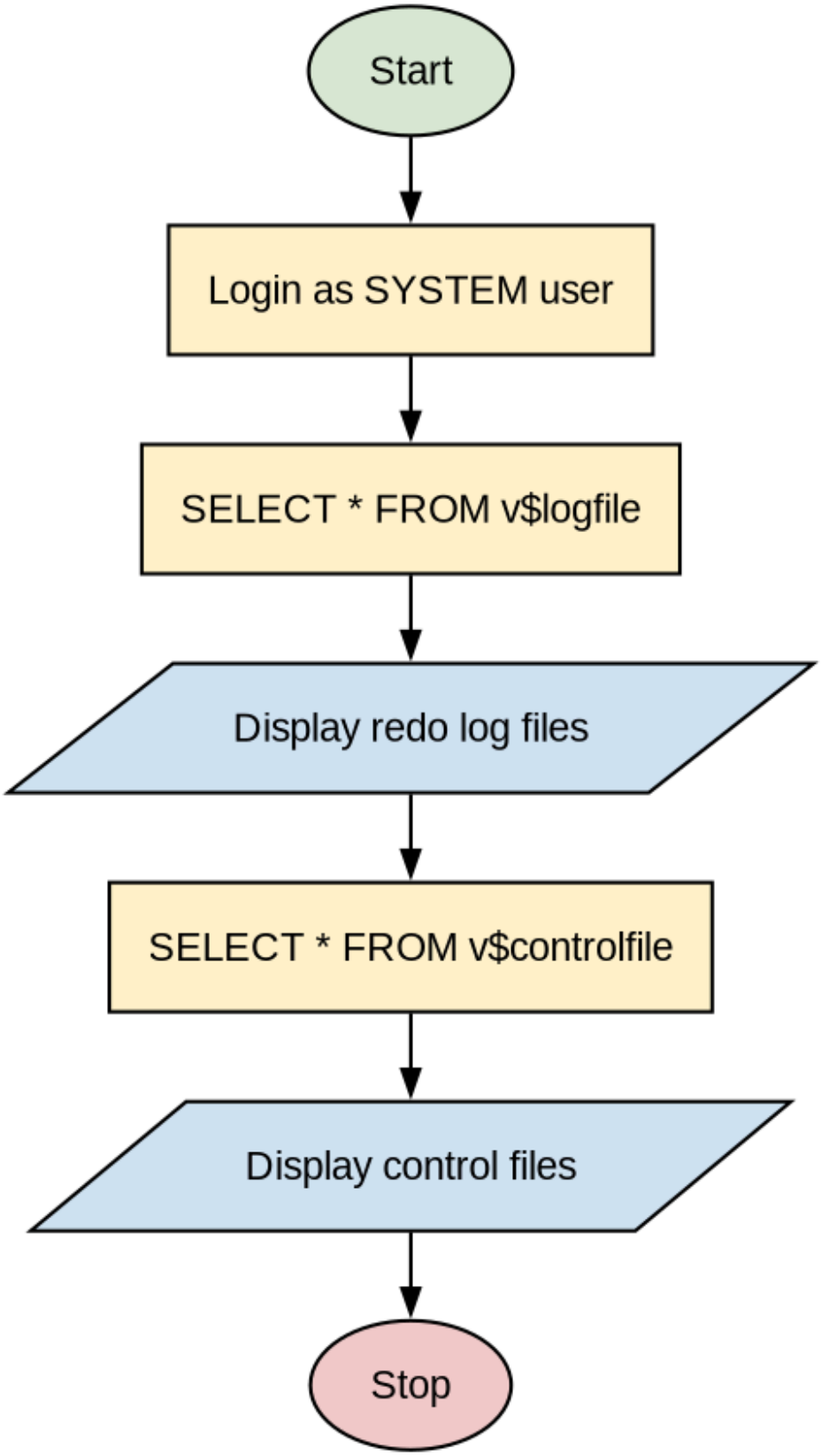
Aim: Display the list of all Redo log files and Control files.

Algorithm:

Step 1: Start.
Step 2: Login as SYSTEM (DBA) user.
Step 3: Query the dynamic view v$logfile to display all redo log file members.
Step 4: Query the dynamic view v$controlfile to display all control files.
Step 5: Stop.

Note: *The output of v$controlfile was not given in the answer sheet; a typical Oracle XE output is added here. Please test and run it yourself before writing.*

Flowchart:


Start
Login as SYSTEM user
SELECT * FROM v$logfile
Display redo log files
SELECT * FROM v$controlfile
Display control files
Stop

Program (SQL Queries):

```
-- List of redo log files
SELECT * FROM v$logfile;

-- List of control files
SELECT * FROM v$controlfile;
```

Output:

```
SQL> select * from v$logfile;

    GROUP# STATUS  TYPE    MEMBER
IS_
---------- ------- ------- ------------------------------------------------------------
-------- ---
         2         ONLINE  C:\ORACLEXE\APP\ORACLE\FLASH_RECOVERY_AREA\XE\ONLINELOG\
                           O1_MF_2_MM8H1MLH_.LOG
YES
         1 STALE   ONLINE  C:\ORACLEXE\APP\ORACLE\FLASH_RECOVERY_AREA\XE\ONLINELOG\
                           O1_MF_1_MM8H1MHF_.LOG
YES

SQL> select * from v$controlfile;

STATUS  NAME                                               IS_ BLOCK_SIZE FILE_SIZE_BLKS
------- -------------------------------------------------- --- ---------- --------------
        C:\ORACLEXE\APP\ORACLE\ORADATA\XE\CONTROL.DBF   NO       16384
594
```

Practical No. 11

Aim: Display the list, datafiles, tablespace names & size in MB.

Algorithm:

Step 1: Start.

Step 2: Login as SYSTEM (DBA) user.

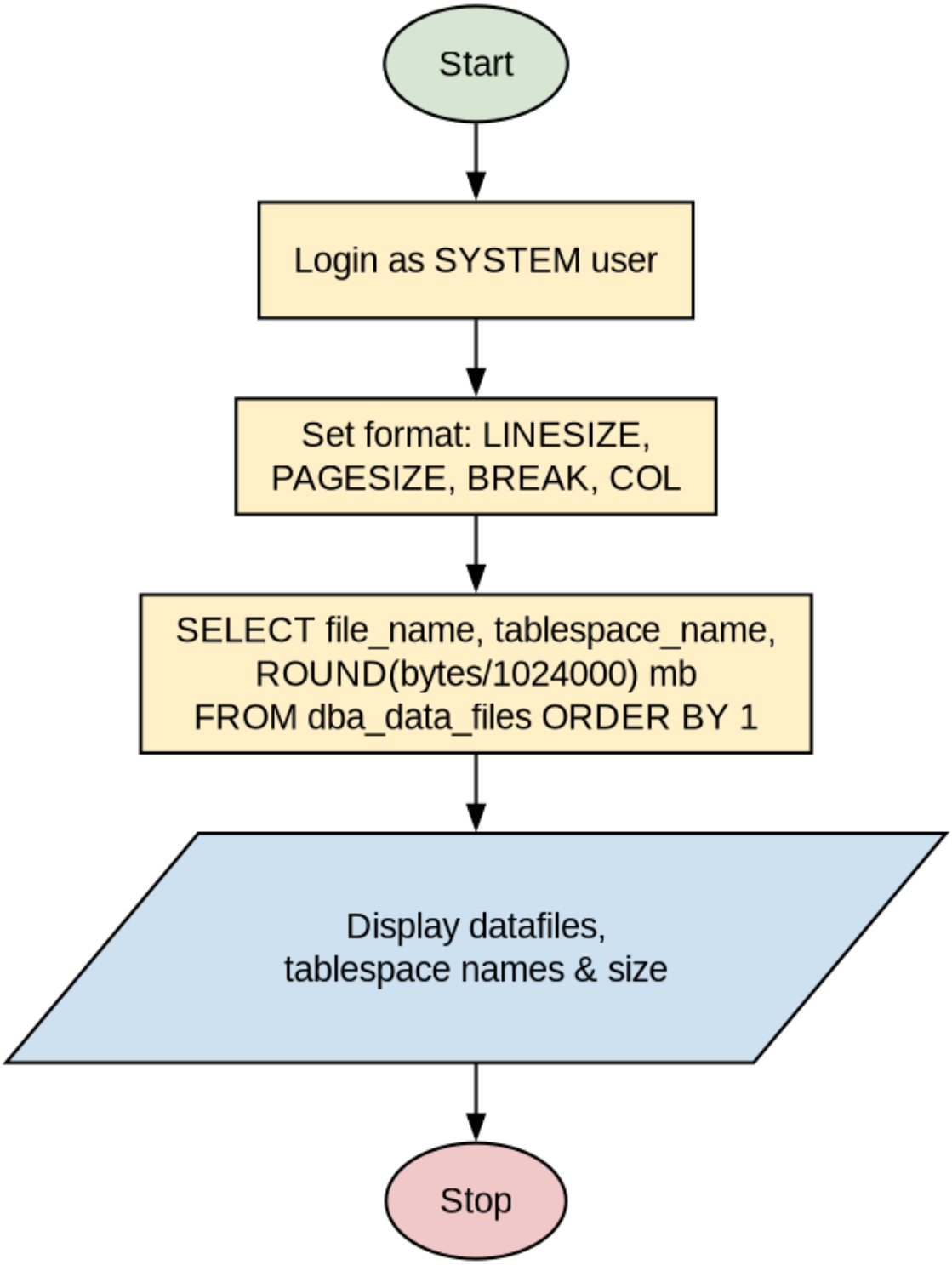
Step 3: Set the SQL*Plus display format using SET LINESIZE, SET PAGESIZE, BREAK and COLUMN commands.

Step 4: Query dba_data_files to select file_name, tablespace_name and size in MB (bytes/1024000).

Step 5: Display the result ordered by file name.

Step 6: Stop.

Flowchart:


Start
Login as SYSTEM user
Set format: LINESIZE, PAGESIZE, BREAK, COL
SELECT file_name, tablespace_name, ROUND(bytes/1024000) mb FROM dba_data_files ORDER BY 1
Display datafiles, tablespace names & size
Stop

Program (SQL Queries):

```
CLEAR BREAKS;
SET LINESIZE 130
SET PAGESIZE 60
BREAK ON tablespace_name SKIP 1
COL tablespace_name FORMAT a15
COL file_name FORMAT a50
COL tablespace_kb HEADING 'TABLESPACE|TOTAL_KB'
COL kbytes_free HEADING 'TOTAL FREE|KBYTES'

SELECT file_name, tablespace_name, ROUND(bytes/1024000) mb
FROM dba_data_files
ORDER BY 1;
```

Output:

```
SQL> select file_name,tablespace_name, ROUND(bytes/1024000) mb
  2  from dba_data_files order by 1;

FILE_NAME                           TABLESPACE_NAME         MB
----------------------------------- --------------- ----------
C:\ORACLEXE\ORADATA\DATAFILE.DBF        TESTING                 10

C:\ORACLEXE\ORADATA\TS_MYDEMO01.DBF     TS_MYDEMO               51
C:\ORACLEXE\ORADATA\TS_MYDEMO02.DBF                             66

C:\ORACLEXE\ORADATA\XE\DATAFILE.DBF     DBA_TABLESPACE          10

C:\ORACLEXE\ORADATA\XE\SYSAUX.DBF       SYSAUX                 451

C:\ORACLEXE\ORADATA\XE\SYSTEM.DBF       SYSTEM                 348

C:\ORACLEXE\ORADATA\XE\UNDO.DBF         UNDO                    92

C:\ORACLEXE\ORADATA\XE\USERS.DBF        USERS                  102

8 rows selected.
```

Practical No. 12

Aim: Create rollback segment in separate tablespace with proper storage option.

Algorithm:

Step 1: Start.

Step 2: Create a separate tablespace small_rollbacks with a datafile of size 20M and DEFAULT STORAGE (INITIAL 250K NEXT 250K MAXEXTENTS 20).

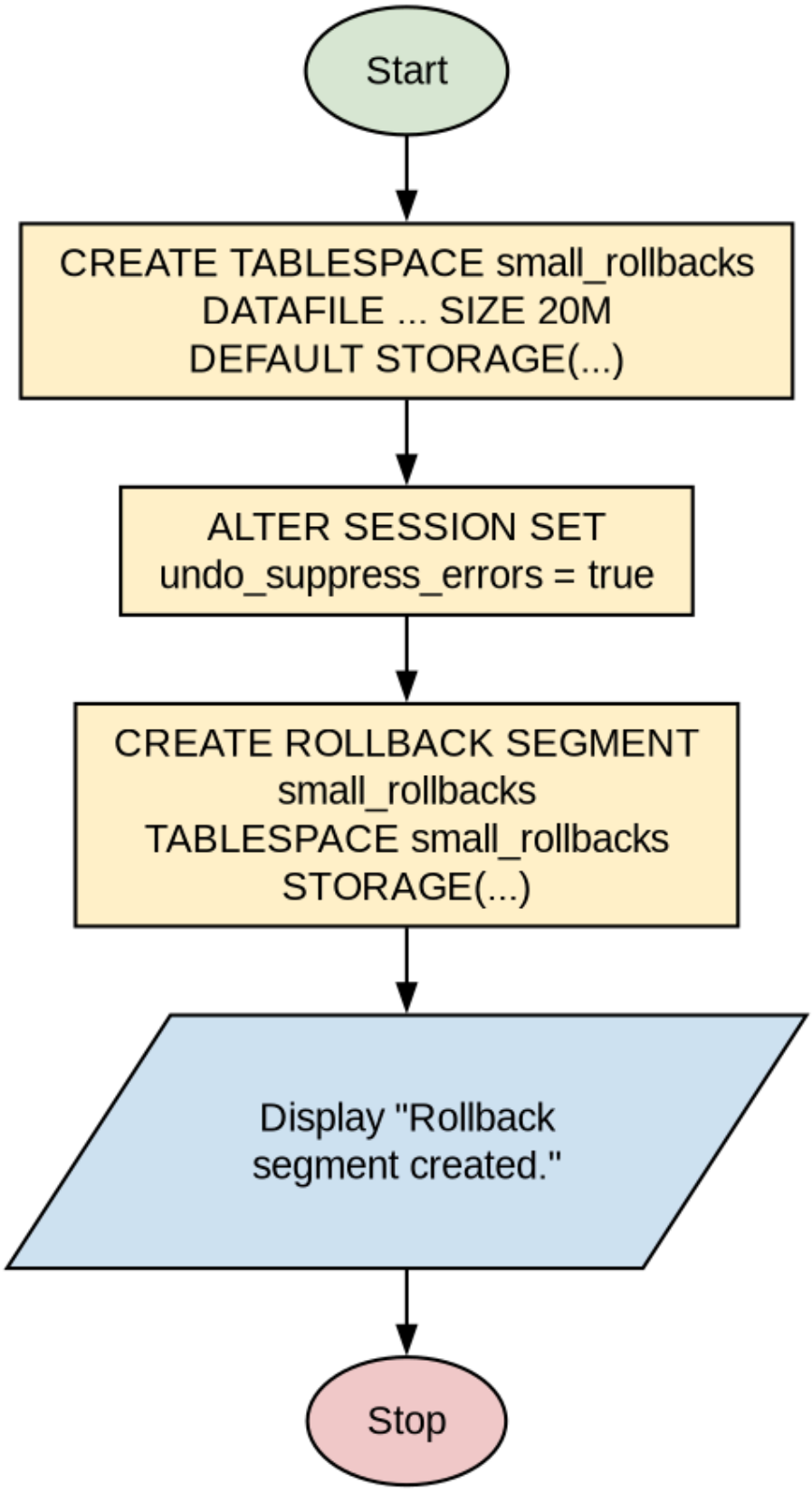
Step 3: Set undo_suppress_errors = true for the session.

Step 4: Create the rollback segment small_rollbacks in tablespace small_rollbacks with the same storage option.

Step 5: Display the message "Rollback segment created."

Step 6: Stop.

Flowchart:


Start
CREATE TABLESPACE small_rollbacks
DATAFILE ... SIZE 20M
DEFAULT STORAGE(...)
ALTER SESSION SET
undo_suppress_errors = true
CREATE ROLLBACK SEGMENT
small_rollbacks
TABLESPACE small_rollbacks
STORAGE(...)
Display "Rollback
segment created."
Stop

Program (SQL Queries):

```
-- Create separate tablespace
CREATE TABLESPACE small_rollbacks
    DATAFILE 'c:program files/oracle/small_rollbacks.dbf' SIZE 20M REUSE
    DEFAULT STORAGE (INITIAL 250K NEXT 250K MAXEXTENTS 20);

ALTER SESSION SET undo_suppress_errors = TRUE;

-- Create rollback segment
CREATE ROLLBACK SEGMENT small_rollbacks
    TABLESPACE small_rollbacks
    STORAGE (INITIAL 250K NEXT 250K MAXEXTENTS 20);
```

Output:

```
SQL> Create tablespace small_rollbacks
  2  datafile 'c:program files/oracle/small_rollbacks.dbf' size 20m reuse
  3* default storage(initial 250k next 250k maxextents 20)

Tablespace created.

SQL> alter session set undo_suppress_errors=true;

Session altered.

SQL> create rollback segment small_rollbacks
  2  tablespace small_rollbacks
  3  storage(initial 250k next 250k maxextents 20);

Rollback segment created.
```

Practical No. 13

Aim: Create a 3 new abstract datatype as given below: 1. NAME_TYPE(F_NAME, M_NAME, L_NAME) 2. POST_TYPE(MAIN, SFX) 3. ADDR_TYPE(STREET, CITY, STATE, POSTCODE)

Algorithm:

Step 1: Start.

Step 2: Create object type name_type with attributes f_name, m_name and l_name using CREATE TYPE ... AS OBJECT.

Step 3: Create object type post_type with attributes main and sfx.

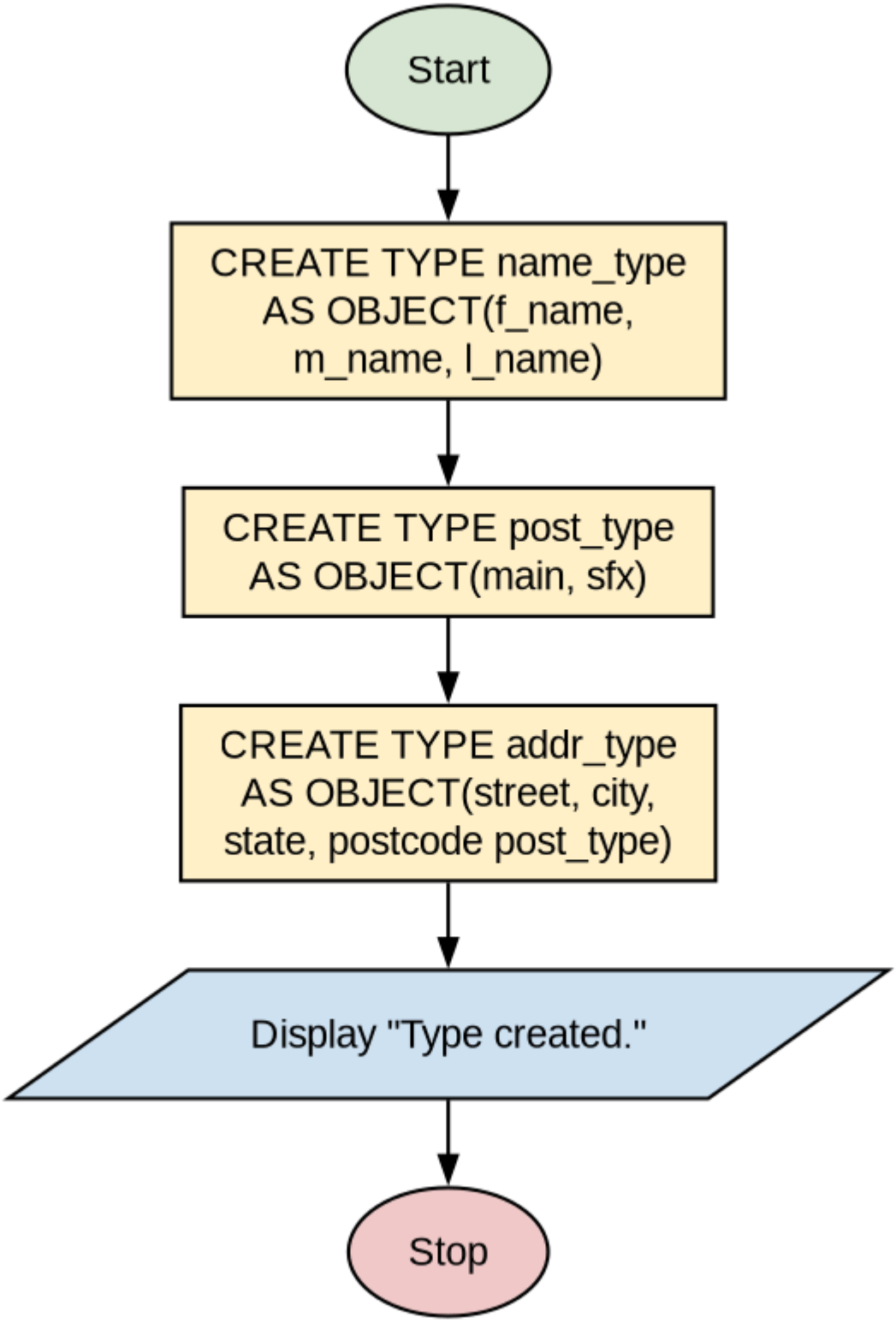
Step 4: Create object type addr_type with attributes street, city, state and postcode, where postcode is of type post_type.

Step 5: Display the message "Type created." for each type.

Step 6: Stop.

Note: *The answer is corrected here: the attribute m_init (answer sheet) is renamed to m_name and add_type is renamed to addr_type, as given in the practical list. Please test and run it yourself before writing.*

Flowchart:


Start
CREATE TYPE name_type AS OBJECT(f_name, m_name, l_name)
CREATE TYPE post_type AS OBJECT(main, sfx)
CREATE TYPE addr_type AS OBJECT(street, city, state, postcode post_type)
Display "Type created."
Stop

Program (SQL Queries):

```
-- 1. Name type
CREATE TYPE name_type AS OBJECT (
    f_name VARCHAR(15),
    m_name CHAR(1),
    l_name VARCHAR2(20)
);
/

-- 2. Post type
CREATE TYPE post_type AS OBJECT (
    main INTEGER,
    sfx  INTEGER
);
/

-- 3. Address type
CREATE TYPE addr_type AS OBJECT (
    street   VARCHAR2(35),
    city     VARCHAR(15),
    state    CHAR(2),
    postcode post_type
);
/
```

Output:

```
SQL> create type name_type as object(f_name varchar(15),
  2  m_name char(1),
  3  l_name varchar2(20));

Type created.

SQL> create type post_type as object(main integer,sfx integer);

Type created.

SQL> create type addr_type as object(
  2  street varchar2(35),
  3  city varchar(15),state char(2),postcode post_type);

Type created.
```

Practical No. 14

Aim: Create a table stud_info having column name, rollno, phy, maths, comp, total and percentage. Write a trigger to automatically calculate students total marks and percentage of students while insertion.

Algorithm:

Step 1: Start.

Step 2: Create table studinfo with columns name, rollno, phy, math, comp, total and per.

Step 3: Create trigger totper BEFORE INSERT ON studinfo FOR EACH ROW.

Step 4: In the trigger body calculate :new.total = :new.math + :new.phy + :new.comp.

Step 5: Calculate :new.per = (:new.total / 300) * 100.

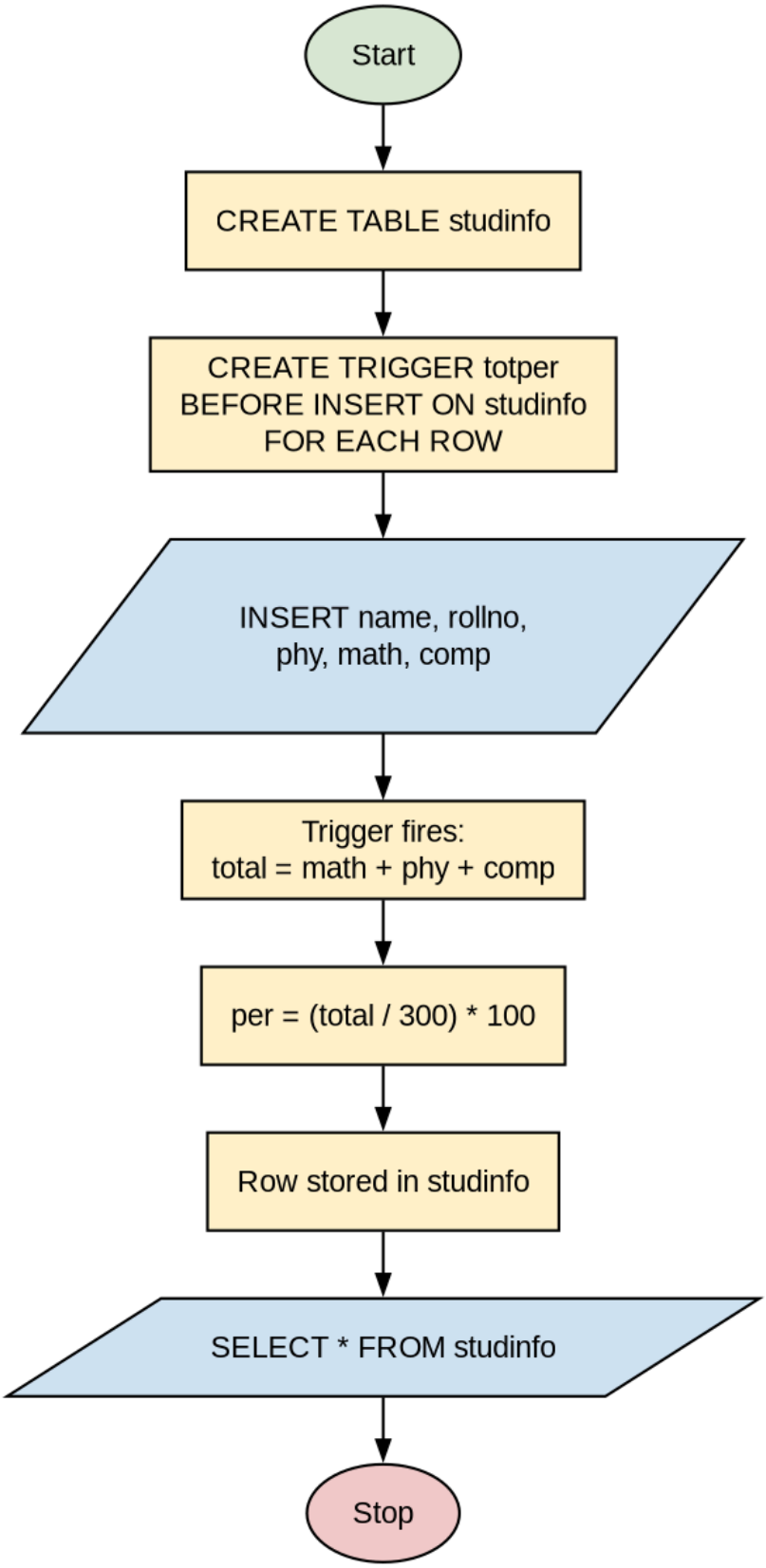
Step 6: Insert a record with only name, rollno, phy, math and comp; the trigger fills total and per automatically.

Step 7: Display the table using SELECT * FROM studinfo.

Step 8: Stop.

Note: *The answer is corrected here: "endtotper;" is written as "end totper;" and the INSERT statement ends with ";". Please test and run it yourself before writing.*

Flowchart:


Start
CREATE TABLE studinfo
CREATE TRIGGER totper
BEFORE INSERT ON studinfo
FOR EACH ROW
INSERT name, rollno,
phy, math, comp
Trigger fires:
total = math + phy + comp
per = (total / 300) * 100
Row stored in studinfo
SELECT * FROM studinfo
Stop

Program (SQL Queries):

```
-- Create table
CREATE TABLE studinfo (
    name   VARCHAR2(20),
    rollno NUMBER(3),
    phy    NUMBER(3),
    math   NUMBER(3),
    comp   NUMBER(3),
    total  NUMBER(5),
    per    NUMBER(3,2)
);

-- Trigger to calculate total and percentage
CREATE OR REPLACE TRIGGER totper
BEFORE INSERT ON studinfo
FOR EACH ROW
BEGIN
    :new.total := :new.math + :new.phy + :new.comp;
    :new.per   := (:new.total / 300) * 100;
END totper;
/

-- Insert record
INSERT INTO studinfo (name, rollno, phy, math, comp)
VALUES ('sid', 1, 7, 7, 7);

-- Display data
SELECT * FROM studinfo;
```

Output:

```
SQL> create table studinfo
  2  (name varchar2(20),
  3  rollno number(3),
  4  phy number(3),
  5  math number(3),
  6  comp number(3),
  7  total number(5),
  8  per number(3,2));

Table created.

SQL> create or replace trigger totper
  2  before insert on studinfo
  3  for each row
  4  begin
  5  :new.total:=:new.math+:new.phy+:new.comp;
  6  :new.per:=(:new.total/300)*100;
  7  end totper;
  8  /

Trigger created.

SQL> insert into studinfo(name,rollno,phy,math,comp)
  2  values('sid',1,7,7,7);

1 row created.

SQL> select * from studinfo;

NAME                      ROLLNO        PHY       MATH       COMP      TOTAL
PER
-------------------- ---------- ---------- ---------- ---------- ---------- --------
--
sid                            1          7          7          7         21
7
```

Practical No. 15

Aim: Write a SQL query to display the following data information: a. Name of database b. Database startup time c. Database size information (database size, used space, free space)

Algorithm:

Step 1: Start.

Step 2: Login as SYSTEM (DBA) user.

Step 3: Display the name of the database from v$database.

Step 4: Display the database startup time from v$instance using TO_CHAR(startup_time, 'hh24:mi dd-mm-yy').

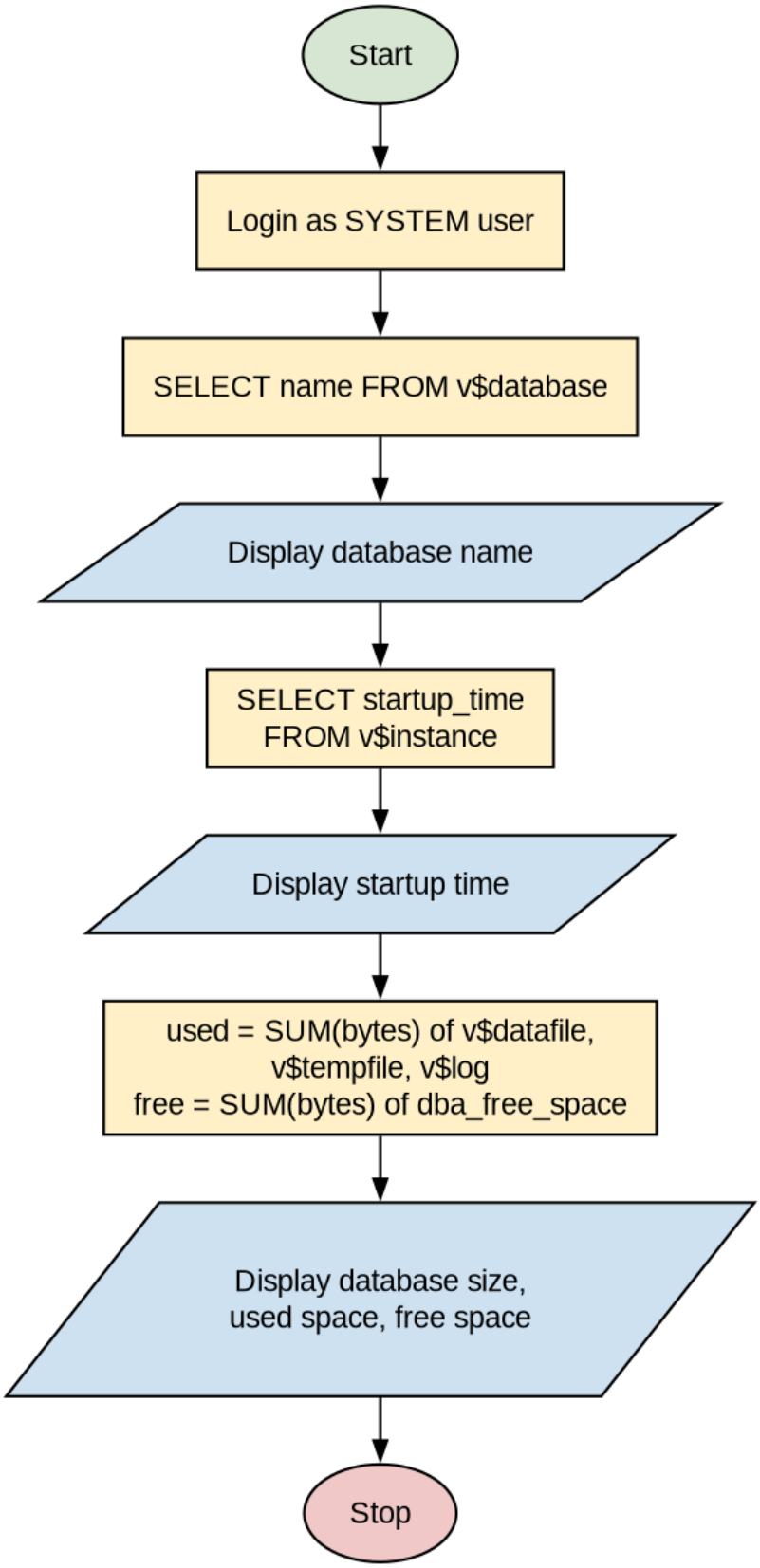
Step 5: Calculate the database size and used space as the sum of bytes from v$datafile, v$tempfile and v$log.

Step 6: Calculate the free space as the sum of bytes from dba_free_space.

Step 7: Display database size (GB), used space (MB) and free space (MB).

Step 8: Stop.

Flowchart:


Start
Login as SYSTEM user
SELECT name FROM v$database
Display database name
SELECT startup_time
FROM v$instance
Display startup time
used = SUM(bytes) of v$datafile,
v$tempfile, v$log
free = SUM(bytes) of dba_free_space
Display database size,
used space, free space
Stop

Program (SQL Queries):

```
-- a. Name of database
SELECT name FROM v$database;

-- b. Database startup time
SELECT TO_CHAR(startup_time, 'hh24:mi dd-mm-yy') "startup time"
FROM v$instance;

-- c. Database size information
SELECT ROUND(SUM(used.bytes)/1024/1024/1024) || 'gb' "database size",
       ROUND(SUM(used.bytes)/1024/1024) || 'mb' "used space",
       ROUND(free.p/1024/1024) || 'mb' "free space"
FROM (SELECT bytes FROM v$datafile
      UNION ALL
      SELECT bytes FROM v$tempfile
      UNION ALL
      SELECT bytes FROM v$log) used,
     (SELECT SUM(bytes) AS p FROM dba_free_space) free
GROUP BY free.p;
```

Output:

```
SQL> select name from v$database;

NAME
---------
ORCL

SQL> select to_char(startup_time,'hh24:mi dd-mm-yy') "startup time"
  2  from v$instance;

startup time
--------------
11:10 06-10-18

SQL> select round(sum(used.bytes)/1024/1024/1024)|| 'gb' "database size",
  2  round(sum(used.bytes)/1024/1024) || 'mb' "used space",
  3  round(free.p/1024/1024) || 'mb' "free space"
  4  from (select bytes
  5  from v$datafile
  6  union all
  7  select bytes
  8  from v$tempfile
  9  union all
 10  select bytes
 11  from v$log) used,
 12  (select sum(bytes) as p
 13  from dba_free_space) free
 14* group by free.p

database size   used space     free space
--------------- -------------- ------------
1gb             1104mb         186mb
```